Action-value methods

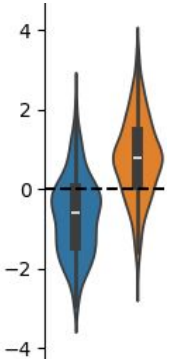
Estimating the value of bandits through experience

What you'll learn

- Understand the exploration vs. exploitation dilemma through methodical walk-throughs of a simple 2-arm problem
- Evaluation: discover action-value methods for estimating bandit rewards through experience
- Control: explore greedy and ε-greedy action-selection strategies balancing exploration and exploitation
- Encounter algorithmic approaches in common with the more complex full RL problem

k-armed bandit // true values

- The true value of an action is the expected reward from taking that action
- Each action has its associated reward distribution
- We are concerned with finding the action which returns the maximum reward over time
- If we had access to the reward distribution we could calculate the expectations and find this action directly
- We typically don't have access to the reward distributions, so need other methods for evaluating actions



$$q_*(0) \doteq \mathbb{E}[R_t | A_t = 0]$$

$$q_*(1) \doteq \mathbb{E}[R_t | A_t = 1]$$

$$a^* = \arg\max_a q_*(a)$$

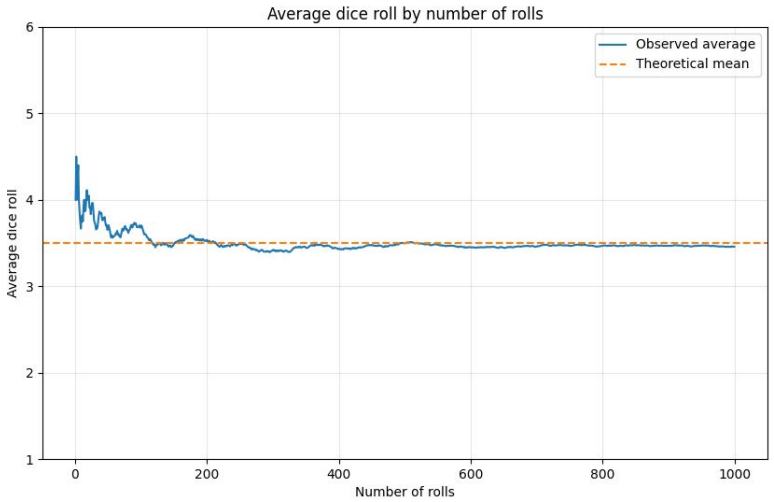
t	0	1	2	3	4	5
$R_t(A_t = 0)$	0.1		-0.8	0.1		0.6
$R_t(A_t = 1)$		-0.5			1.8	

Value estimate (evaluation)

$$Q_t(a) \doteq \frac{\text{sum of rewards when } a \text{ taken prior to } t}{\text{number of times } a \text{ taken prior to } t} = \frac{\sum_{i=1}^{t-1} R_i \cdot \mathbf{1}_{A_i=a}}{\sum_{i=1}^{t-1} \mathbf{1}_{A_i=a}},$$

where $\mathbf{1}_{\text{predicate}}$ is an indicator function:

$$\mathbf{1}_{\text{predicate}} = \begin{cases} 1 & \text{if predicate is true} \\ 0 & \text{otherwise} \end{cases}$$



Value estimate (evaluation)

- Empirical value estimation: Calculate the average reward received for each action, updating continuously as more actions are taken

- Convergence via Law of Large Numbers: With enough trials, the estimate stabilises, converging towards the true expected value

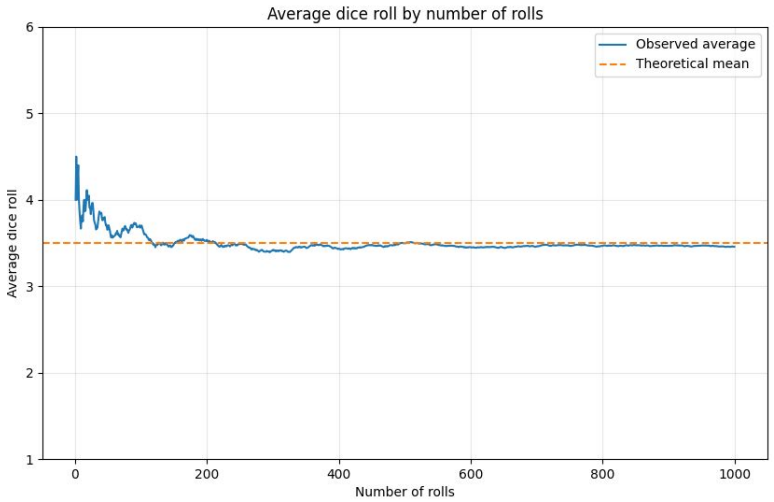
- A "Monte Carlo" approach: This method is a form of Monte Carlo estimation, where values are learned from sampled experiences
  - We will explore more sophisticated Monte Carlo techniques later in the course

- Note on notation:
  - $Q_t(a)$ (upper-case) - our value estimate is a random variable, which varies as we collect more data
    $q_*(a)$ (lower-case) - is deterministic and represents the true expected reward for action a (which unfortunately we can't typically calculate as we lack access to the reward distribution)

$$Q_t(a) \doteq \frac{\text{sum of rewards when } a \text{ taken prior to } t}{\text{number of times } a \text{ taken prior to } t} = \frac{\sum_{i=1}^{t-1} R_i \cdot \mathbf{1}_{A_i=a}}{\sum_{i=1}^{t-1} \mathbf{1}_{A_i=a}},$$

where $\mathbf{1}_{\text{predicate}}$ is an indicator function:

$$\mathbf{1}_{\text{predicate}} = \begin{cases} 1 & \text{if predicate is true} \\ 0 & \text{otherwise} \end{cases}$$



Action selection (control) // greedy

- Pull each lever once at the start
- (Evaluation) update the estimated value function on each pull
- (Control) greedily select the action with the maximum value estimate
- Note on the evaluation/control distinction
  - Evaluation (or similarly, prediction) is the act of calculating or estimating the value function
  - Control is the act of using this value function to improve the actions we select
  - **This division of calculation steps will be maintained throughout future reinforcement learning lectures**

$$a^* = \arg\max_a q_*(a)$$

$$Q_t(a) \doteq \frac{\text{sum of rewards when } a \text{ taken prior to } t}{\text{number of times } a \text{ taken prior to } t} = \frac{\sum_{i=1}^{t-1} R_i \cdot \mathbf{1}_{A_i=a}}{\sum_{i=1}^{t-1} \mathbf{1}_{A_i=a}},$$

$$A_t = \arg\max_a Q_t(a)$$

Action selection (control) // greedy

- Pull each lever once at the start
- (Evaluation) update the estimated value function on each pull
- (Control) greedily select the action with the maximum value estimate
- Note on the evaluation/control distinction
  - Evaluation (or similarly, prediction) is the act of calculating or estimating the value function
  - Control is the act of using this value function to improve the actions we select
  - **This division of calculation steps will be maintained throughout future reinforcement learning lectures**

Action 0	Action 1
$r(0) \sim \mathcal{N}(0, 2^2)$	$r(1) \sim \mathcal{N}(1, 1^2)$
$q_*(0) = 0$	$q_*(1) = 1$

t	0	1	2	3	4	5	6	7	8	9
Q_t(0)	0									
Q_t(1)	0									
(a = 0) selected	✓									
(a = 1) selected										
R_t										

Action selection (control) // greedy

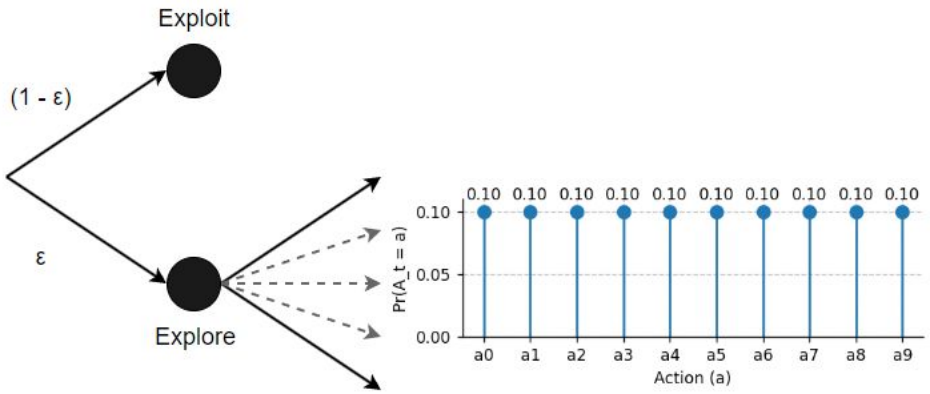
- Pull each lever once at the start
- (Evaluation) update the estimated value function on each pull
- (Control) greedily select the action with the maximum value estimate
- Note on the evaluation/control distinction
  - Evaluation (or similarly, prediction) is the act of calculating or estimating the value function
  - Control is the act of using this value function to improve the actions we select
  - **This division of calculation steps will be maintained throughout future reinforcement learning lectures**

Action 0	Action 1
$r(0) \sim \mathcal{N}(0, 2^2)$	$r(1) \sim \mathcal{N}(1, 1^2)$
$q_*(0) = 0$	$q_*(1) = 1$

t	0	1	2	3	4	5	6	7	8	9
Q_t(0)	0	3	3	1.7	1.47	0.85	0.58	0.57	0.49	0.49
Q_t(1)	0	0	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.4
(a = 0) selected	✓		✓	✓	✓	✓	✓	✓		✓
(a = 1) selected		✓							✓	
R_t	3	0.5	0.4	1	-1	-0.5	0.5	0	0.3	...

Action selection // ε-greedy

$$A_t \leftarrow \begin{cases} \arg\max_a Q(a) & \text{with probability } 1-\varepsilon \quad \text{(breaking ties randomly)} \\ \text{a random action} & \text{with probability } \varepsilon \end{cases}$$



$$\pi(a) = \Pr(A_t = a) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{|\mathcal{A}|}, & \text{if } a = \arg\max_{a'} Q_t(a') \text{ (exploit)} \\ \frac{\epsilon}{|\mathcal{A}|}, & \text{if } a \neq \arg\max_{a'} Q_t(a') \text{ (explore)} \end{cases}$$

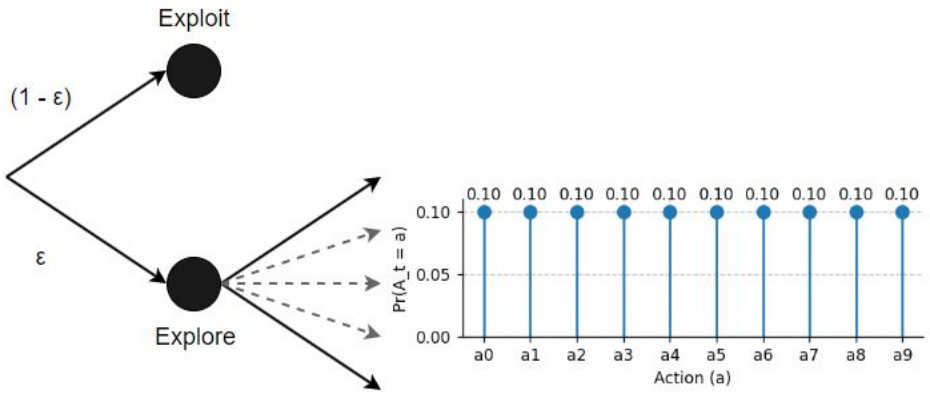
where $\mathcal{A}$ is the set of all possible actions.

Action selection // ε-greedy

- **Greedy action selection can get stuck making sub-optimal decisions**
  - We saw this in our simple walk-through example

$$A_t \leftarrow \begin{cases} \arg\max_a Q(a) & \text{with probability } 1-\varepsilon \quad \text{(breaking ties randomly)} \\ \text{a random action} & \text{with probability } \varepsilon \end{cases}$$

- **ε-greedy action selection:**
  - a small fraction (ε) of the time, take an exploratory action
  - The (majority) remainder of the time, exploit our current estimate of the best action (greedy)
  - E.g. ε = 0.25 implies a quarter of the time, take a random action



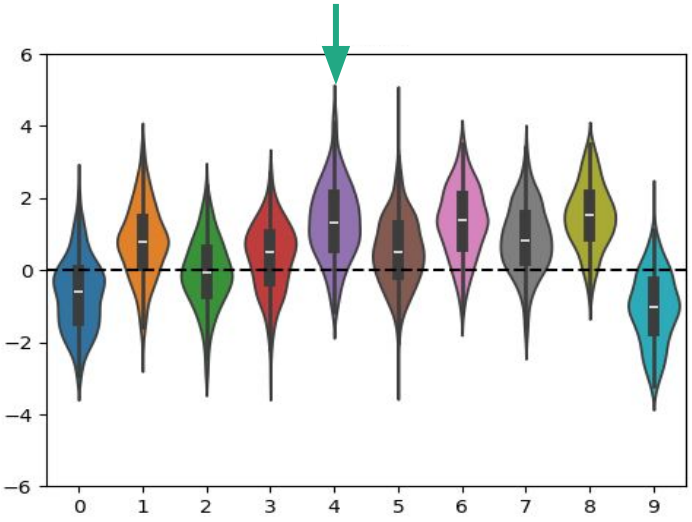
- **Note randomness as a consequence of randomness:**
  - When exploratory option selected with probability ε, the action is then also selected at random
  - This is typically via sampling from a uniform distribution

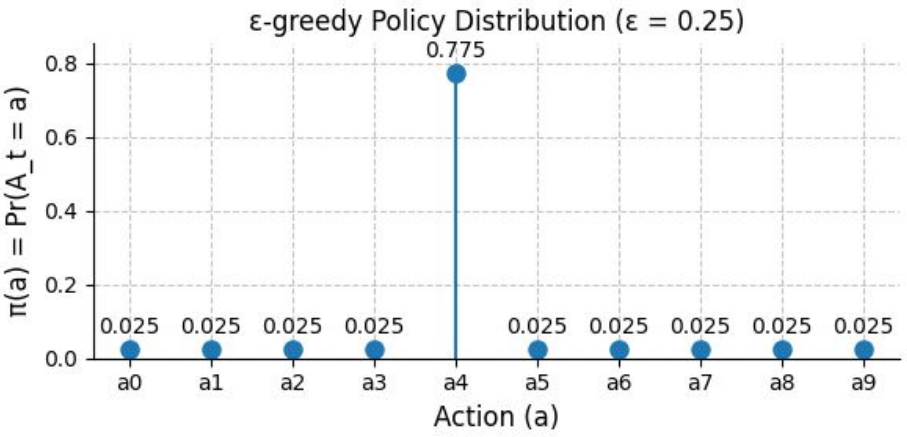
- **A policy (π) is our probability distribution for selecting actions**
  - Greedy policies are deterministic
  - ε-greedy policies are stochastic

$$\pi(a) = \Pr(A_t = a) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{|\mathcal{A}|}, & \text{if } a = \arg\max_{a'} Q_t(a') \text{ (exploit)} \\ \frac{\epsilon}{|\mathcal{A}|}, & \text{if } a \neq \arg\max_{a'} Q_t(a') \text{ (explore)} \end{cases}$$

where $\mathcal{A}$ is the set of all possible actions.

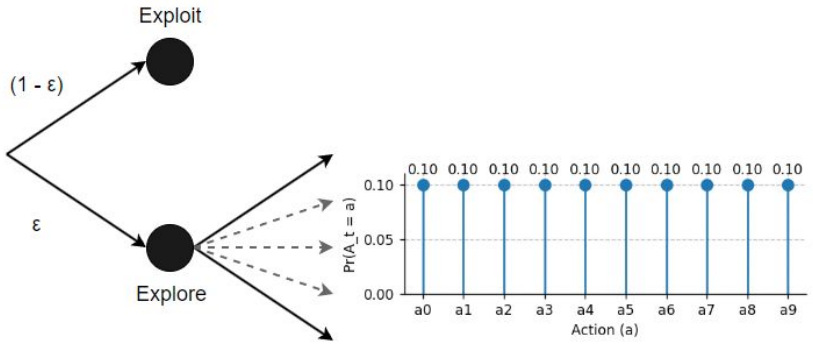
Action selection // ε-greedy





ε-greedy // implementation

$$A_t \leftarrow \begin{cases} \arg\max_a Q(a) & \text{with probability } 1-\varepsilon \quad \text{(breaking ties randomly)} \\ \text{a random action} & \text{with probability } \varepsilon \end{cases}$$



$$\pi(a) = \Pr(A_t = a) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{|\mathcal{A}|}, & \text{if } a = \arg\max_{a'} Q_t(a') \text{ (exploit)} \\ \frac{\epsilon}{|\mathcal{A}|}, & \text{if } a \neq \arg\max_{a'} Q_t(a') \text{ (explore)} \end{cases}$$

where $\mathcal{A}$ is the set of all possible actions.

```python
class EpsilonGreedy:

    def __init__(self, env, epsilon, max_steps=1000, initialisation=0):

        self.env = env
        self.epsilon = epsilon
        self.max_steps = max_steps
        self.num_actions = env.bandits[0].k    # Number of bandits in any of the k-armed bandits within the testbed
        self.q_values = # … initialise etc
        ...
```

```python
    def act(self):

        if np.random.random() < self.epsilon:
            return np.random.randint(0, self.num_actions)
        else:
            return argmax_ties_random(self.q_values)
```

```python
    def compute_probs(self):
        probs = np.ones(self.num_actions) * self.epsilon / self.num_actions
        best_action = argmax_ties_random(self.q_values)
        probs[best_action] =1 - self.epsilon + self.epsilon / self.num_actions
```

ε-greedy // walk-through

Action 0	Action 1
$r(0) \sim \mathcal{N}(0, 2^2)$	$r(1) \sim \mathcal{N}(1, 1^2)$
$q_*(0) = 0$	$q_*(1) = 1$

$$a^* = \arg\max_a q_*(a) = \arg\max_a \begin{pmatrix} 0 \\ 1 \end{pmatrix} = 1$$

greedy

t	0	1	2	3	4	5	6	7	8	9
Q_t(0)	0	3	3	1.7	1.47	0.85	0.58	0.57	0.49	0.49
Q_t(1)	0	0	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.4
(a = 0) selected	✓		✓	✓	✓	✓	✓	✓		✓
(a = 1) selected		✓							✓	
R_t	3	0.5	0.4	1	-1	-0.5	0.5	0	0.3	...

ε-greedy

t	0	1	2	3
Q_t(0)	0	3	3	1.7
Q_t(1)	0	0	0.5	0.5
(a = 0) selected	✓		✓	✓
(a = 1) selected		✓		
R_t	3	0.5	0.4	1

ε-greedy // walk-through

Action 0	Action 1
$r(0) \sim \mathcal{N}(0, 2^2)$	$r(1) \sim \mathcal{N}(1, 1^2)$
$q_*(0) = 0$	$q_*(1) = 1$

$$a^* = \arg\max_a q_*(a) = \arg\max_a \begin{pmatrix} 0 \\ 1 \end{pmatrix} = 1$$

greedy

t	0	1	2	3	4	5	6	7	8	9
Q_t(0)	0	3	3	1.7	1.47	0.85	0.58	0.57	0.49	0.49
Q_t(1)	0	0	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.4
(a = 0) selected	✓		✓	✓	✓	✓	✓	✓		✓
(a = 1) selected		✓							✓	
R_t	3	0.5	0.4	1	-1	-0.5	0.5	0	0.3	...

ε-greedy

t	0	1	2	3	4	5	6	7	8	9
Q_t(0)	0	3	3	1.7	1.47	1.47	0.98	0.98	0.98	0.98
Q_t(1)	0	0	0.5	0.5	0.5	1.25	1.25	1.17	1.25	1.1
(a = 0) selected	✓		✓	✓		✓				
(a = 1) selected		✓			✓		✓	✓	✓	✓
R_t	3	0.5	0.4	1	2	-0.5	1	1.5	0.5	...
					Explore				Explore	

Calculating the sample mean // basic

ε-greedy

t	0	1	2	3	4	5	6	7	8	9
Q_t(0)	0	3	3	1.7	1.47	1.47	0.98	0.98	0.98	0.98
Q_t(1)	0	0	0.5	0.5	0.5	1.25	1.25	1.17	1.25	1.1
(a = 0) selected	✓		✓	✓		✓				
(a = 1) selected		✓			✓		✓	✓	✓	✓
R_t	3	0.5	0.4	1	2	-0.5	1	1.5	0.5	...
					Explore				Explore	

curbar.ai

Calculating the sample mean // basic

$$Q_t(a) \doteq \frac{\text{sum of rewards when } a \text{ taken prior to } t}{\text{number of times } a \text{ taken prior to } t} = \frac{\sum_{i=1}^{t-1} R_i \cdot \mathbf{1}_{A_i=a}}{\sum_{i=1}^{t-1} \mathbf{1}_{A_i=a}}$$

$$Q_n = \frac{1}{n-1}\sum_{i=1}^{n-1} R_i \qquad \text{(Single action; } n \text{ rather than } t \text{ to index)}$$

$$Q_{n+1} = \frac{1}{n}\sum_{i=1}^{n} R_i \qquad \text{(Form for calculation post reward)}$$

ε-greedy

t	0	1	2	3	4	5	6	7	8	9
Q_t(0)	0	3	3	1.7	1.47	1.47	0.98	0.98	0.98	0.98
Q_t(1)	0	0	0.5	0.5	0.5	1.25	1.25	1.17	1.25	1.1
(a = 0) selected	✓		✓	✓		✓				
(a = 1) selected		✓			✓		✓	✓	✓	✓
R_t	3	0.5	0.4	1	2	-0.5	1	1.5	0.5	...
					Explore				Explore	

t	$rewards_t(A_t = 0)$	$rewards_t(A_t = 1)$	$Q_{t+1}(0)$	$Q_{t+1}(1)$
<init>	[]	[]	0	0
0	[3]	[]	3	0
1	[3]	[0.5]	3	0.5
2	[3, 0.4]	[0.5]	1.7	0.5
3	[3, 0.4, 1]	[0.5]	1.47	0.5
4	[3, 0.4, 1]	[0.5, 2]	1.47	1.25
5	[3, 0.4, 1, -0.5]	[0.5, 2]	0.98	1.25
6	[3, 0.4, 1, -0.5]	[0.5, 2, 1]	0.98	1.17
7	[3, 0.4, 1, -0.5]	[0.5, 2, 1, 1.5]	0.98	1.25
8	[3, 0.4, 1, -0.5]	[0.5, 2, 1, 1.5, 0.5]	0.98	1.1

Calculating the sample mean // incremental

- **Reformulated the sample average in a way that the updated value estimate depends solely on:**
  - The previous value estimate
  - The newly received reward
  - An incremented count
  - This is an **incremental update equation**

Incremental approach - **complexity**

- **Space complexity - O(1)**
  - Q_n, R_n and n are single values
- **Time complexity per update - O(1)**
  - The update equation is a fixed combination of terms

Basic approach - **complexity**

- **Space complexity - O(n)**
  - $Rewards_n(a)$ is a list of numbers of length n for action a
- **Time complexity per update - O(n)**
  - Averaging a list = sum(list) / len(list)
  - sum(list) is O(n)
  - len(list) is O(1) (python implements this efficiently)
  - O(n) term dominates

Calculating the sample mean // incremental

$$
\begin{aligned}
Q_{n+1} &= \frac{1}{n}\sum_{i=1}^{n} R_i && \text{(Sample average definition)} \\
&= \frac{1}{n}\left(R_n + \sum_{i=1}^{n-1} R_i\right) && \text{(Separate latest reward)} \\
&= \frac{1}{n}\left(R_n + (n-1)\frac{1}{n-1}\sum_{i=1}^{n-1} R_i\right) && \text{(Factor out (n-1))} \\
&= \frac{1}{n}\left(R_n + (n-1)Q_n\right) && \text{(Recognise } Q_n\text{)} \\
&= \frac{1}{n}\left(R_n + nQ_n - Q_n\right) && \text{(Expand } (n-1)Q_n\text{)} \\
&= \boxed{Q_n + \frac{1}{n}\left[R_n - Q_n\right].} && \text{(Rearrange terms)}
\end{aligned}
$$

$$NewEstimate \leftarrow OldEstimate + StepSize[Target - OldEstimate]$$
$$NewEstimate \leftarrow OldEstimate + StepSize[Error]$$

- **Reformulated the sample average in a way that the updated value estimate depends solely on:**
  - The previous value estimate
  - The newly received reward
  - An incremented count
  - This is an **incremental update equation**

Incremental approach - complexity

- **Space complexity - O(1)**
  - Q_n, R_n and n are single values
- **Time complexity per update - O(1)**
  - The update equation is a fixed combination of terms

Basic approach - complexity

- **Space complexity - O(n)**
  - *Rewards*$_n$(a) is a list of numbers of length n for action a
- **Time complexity per update - O(n)**
  - Averaging a list = sum(list) / len(list)
  - sum(list) is O(n)
  - len(list) is O(1) (python implements this efficiently)
  - O(n) term dominates

Calculating the mean // incremental

$$Q_{n+1}(a) = Q_n(a) + \frac{1}{n(a)}\left[R_n(a) - Q_n(a)\right]$$

$$NewEstimate \leftarrow OldEstimate + StepSize[Target - OldEstimate]$$

ε-greedy

t	0	1	2	3	4	5	6	7	8	9
Q_t(0)	0	3	3	1.7	1.47	1.47	0.98	0.98	0.98	0.98
Q_t(1)	0	0	0.5	0.5	0.5	1.25	1.25	1.17	1.25	1.1
(a = 0) selected	✓		✓	✓		✓				
(a = 1) selected		✓			✓		✓	✓	✓	✓
R_t	3	0.5	0.4	1	2	-0.5	1	1.5	0.5	...
					Explore				Explore	

curbar.ai

Calculating the mean // incremental

$$Q_{n+1}(a) = Q_n(a) + \frac{1}{n(a)}\left[R_n(a) - Q_n(a)\right]$$

$$NewEstimate \leftarrow OldEstimate + StepSize[Target - OldEstimate]$$

ε-greedy

t	0	1	2	3	4	5	6	7	8	9
Q_t(0)	0	3	3	1.7	1.47	1.47	0.98	0.98	0.98	0.98
Q_t(1)	0	0	0.5	0.5	0.5	1.25	1.25	1.17	1.25	1.1
(a = 0) selected	✓		✓	✓		✓				
(a = 1) selected		✓			✓		✓	✓	✓	✓
R_t	3	0.5	0.4	1	2	-0.5	1	1.5	0.5	...
					Explore				Explore	

t	R_t	A_t	N(0)	N(1)	Q_t+1(0)	Q_t+1(1)
<init>			0	0	0	0
0	3	0	1	0	3	0
1	0.5	1	1	1	3	0.5
2	0.4	0	2	1	1.7	0.5
3	1	0	3	1	1.47	0.5
4	2	1	3	2	1.47	1.25
5	-0.5	0	4	2	0.98	1.25
6	1	1	4	3	0.98	1.17
7	1.5	1	4	4	0.98	1.25
8	0.5	1	4	5	0.98	1.1

ε-greedy strategy

Algorithm 1 A simple bandit algorithm

Initialize, for $a = 1$ to k:
  $Q(a) \leftarrow 0$
  $N(a) \leftarrow 0$
Loop forever:
  $A \leftarrow \begin{cases} \arg\max_a Q(a) & \text{with probability } 1-\varepsilon \text{ (breaking ties randomly)} \\ \text{a random action} & \text{with probability } \varepsilon \end{cases}$
  $R \leftarrow \text{bandit}(A)$
  $N(A) \leftarrow N(A) + 1$
  $Q(A) \leftarrow Q(A) + \frac{1}{N(A)}[R - Q(A)]$

ε-greedy strategy

- Initialise algorithm variables
  - Q(a) and N(a) both zero arrays of length k
- Infinite loop
  - E.g. the instagram feed example never stops
  - A more complex algorithm could include convergence break points based on e.g. confidence values
- Control: ε-greedy action selection
- "Environment" response - bandit array returns a reward
- Increment count
- Evaluation: efficient update of value estimate for selected arm

Algorithm 1 A simple bandit algorithm

Initialize, for $a = 1$ to k:
  $Q(a) \leftarrow 0$
  $N(a) \leftarrow 0$
Loop forever:
  $A \leftarrow \begin{cases} \arg\max_a Q(a) & \text{with probability } 1-\varepsilon \text{ (breaking ties randomly)} \\ \text{a random action} & \text{with probability } \varepsilon \end{cases}$
  $R \leftarrow \text{bandit}(A)$
  $N(A) \leftarrow N(A) + 1$
  $Q(A) \leftarrow Q(A) + \frac{1}{N(A)}[R - Q(A)]$

Summary & wrap-up

- Mastered the basics of k-armed bandits: from greedy to ε-greedy approaches
- Unravelled the explore-exploit trade-off: balancing immediate gains with long-term learning
- Grasped the efficiency of incremental updates: saving space and time in value estimation
- Set the foundation for more advanced reinforcement learning concepts to come